Graph Traversal Algorithms

DFS & BFS — Complete Theory + Code Guide

Stack / Queue	Python + C	All Variants
Both Variants	Code for Both	Recursive & Iterative

Covers: DFS (Stack · Recursive · Matrix · List) | BFS (Queue · Recursive · Matrix · List) | Dry Runs | Comparisons

1. Quick Graph Refresher

Before diving into traversals, make sure these concepts are solid.

1.1 What Is a Graph?

A Graph is a collection of Vertices (nodes) connected by Edges (lines/arrows). Unlike a tree, a graph can have cycles, disconnected parts, and edges going in any direction.

Key Terms

Vertex (node): a point in the graph (e.g. city, person, webpage)
Edge: a connection between two vertices (e.g. road, friendship, hyperlink)
Directed Graph: edges have a direction (1 → 2 means you can go FROM 1 TO 2 only)
Undirected Graph: edges go both ways (1 — 2 means 1↔2)
Weighted Graph: edges have a cost/distance (1 --5km-- 2)
Adjacent: two vertices directly connected by an edge
Degree: number of edges connected to a vertex

1.2 Two Ways to Store a Graph in Memory

Adjacency Matrix

A 2D grid of size N×N where N = number of vertices. mat[src][dest] = 1 means an edge exists.

```
Graph with 4 vertices (0,1,2,3) and edges: 0-1, 1-2, 2-3

     0   1   2   3
  0 [0] [1] [0] [0]
  1 [1] [0] [1] [0]
  2 [0] [1] [0] [1]
  3 [0] [0] [1] [0]

mat[0][1] = 1  →  edge 0→1 exists
mat[0][2] = 0  →  no edge between 0 and 2
```

Adjacency Matrix — Pros	Adjacency Matrix — Cons
O(1) to check if edge exists	O(V²) space — wastes memory for sparse graphs
Simple to implement	Iterating neighbors takes O(V) even if few
Best for Dense Graphs (many edges)	Bad for Sparse Graphs (few edges)

Adjacency List

Each vertex stores a list of its neighbors. Far more memory-efficient for sparse graphs.

```
Same graph: 0-1, 1-2, 2-3
```

```
adj[0] = [1]
adj[1] = [0, 2]
adj[2] = [1, 3]
adj[3] = [2]

Python: adj = {0:[1], 1:[0,2], 2:[1,3], 3:[2]}
Or:     adj = [[] for _ in range(n)]
        adj[0].append(1); adj[1].append(0) ...
```

Adjacency List — Pros	Adjacency List — Cons
O(V+E) space — very efficient	O(degree) to check if specific edge exists
Fast neighbor iteration	Slightly more complex to implement
Best for Sparse Graphs	Edge check not O(1) like matrix

2. Depth-First Search (DFS) — Complete Guide

DFS in One Line
Go as DEEP as possible down one path before backtracking and trying another path.
Think of it as exploring a maze — keep going forward until you hit a dead end, then backtrack.

2.1 The Core Idea — How DFS Thinks

Imagine you are in a maze. DFS says: pick ANY direction and walk as far as you can. When you hit a dead end (visited everything or no more neighbors), BACKTRACK to the last junction and try a different direction. Keep doing this until every room is visited.

```
Graph:                      DFS from 0:
  0                         Visit 0 → go deep to 1
 / \                        Visit 1 → go deep to 3
1   2                       Visit 3 → dead end, backtrack
|   |                       Backtrack to 1 → already explored
3   4                       Backtrack to 0 → try 2
                            Visit 2 → go deep to 4
                            Visit 4 → done

DFS Order: 0 → 1 → 3 → 2 → 4
```

2.2 DFS — Two Implementations

DFS can be implemented in two ways: using an explicit Stack (iterative), or using Recursion (which uses the call stack internally). Both produce the same result.

Iterative (Explicit Stack)	Recursive (Call Stack)
Uses a stack data structure you manage yourself	Python/C handles the stack via function calls
stack = [src] → push neighbors → pop → repeat	dfs(node) calls dfs(neighbor) inside itself
Good for very deep graphs (no stack overflow)	Cleaner, shorter code
Slightly harder to read	Can hit recursion limit on huge graphs

2.3 DFS — Iterative (Stack) with Adjacency Matrix (Python)

This is the version from your class. The graph is stored as a matrix.

```
from collections import deque

class Graph:
    def __init__(self, vertex):
        # Create N x N matrix filled with 0s
        # [0]*vertex = [0,0,0,...] (one row)
        # for x in range(vertex) = repeat that row 'vertex' times
```

```
        self.mat = [[0]*vertex for x in range(vertex)]
        self.size = vertex

    def add_edge(self, src, dest):
        # Boundary check: valid indices only
        if (0 <= src < self.size) and (0 <= dest < self.size):
            self.mat[src][dest] = 1   # src → dest
            self.mat[dest][src] = 1   # dest → src (undirected)
        else:
            print('Invalid Edge')

    def dfs(self, src):
        visited = [False] * self.size   # track visited nodes
        stack = [src]                   # start: push source

        while stack:                    # keep going while stack not empty
            v = stack.pop()             # pop from TOP (LIFO)

            if visited[v] == False:     # only process if not yet visited
                print(v, end=' -> ')    # print this node
                visited[v] = True       # mark as visited

                # Check all potential neighbors of v
                for i in range(self.size):
                    # mat[v][i]==1 means edge v→i exists
                    # not visited[i] means we haven't seen it
                    if self.mat[v][i] == 1 and visited[i] == False:
                        stack.append(i)  # push neighbor to stack

# Usage:
g = Graph(6)
g.add_edge(0,1); g.add_edge(0,2)
g.add_edge(2,3); g.add_edge(2,4)
g.add_edge(3,5); g.add_edge(4,5)
g.dfs(0)  # Output: 0 -> 2 -> 4 -> 5 -> 3 -> 1 ->
```

Why mark visited when POPPING (not pushing) in iterative DFS?
In iterative DFS, the same node can be pushed multiple times (by different neighbors) before it is ever popped. Checking visited[v] == False when we POP ensures we only process it once, even if it was pushed 3 times. This is safe because: once it's popped and printed, we set visited[v] = True, and any future pops of that same node will fail the if-check and be skipped harmlessly.

2.4 DFS — Iterative with Adjacency LIST (Python)

More efficient for sparse graphs. Instead of checking all N columns, we only look at actual neighbors.

```
class GraphList:
    def __init__(self, vertex):
        self.size = vertex
        # Each vertex gets its own list of neighbors
        self.adj = [[] for _ in range(vertex)]

    def add_edge(self, src, dest):
```

```
        self.adj[src].append(dest)   # src → dest
        self.adj[dest].append(src)   # dest → src (undirected)

    def dfs(self, src):
        visited = [False] * self.size
        stack = [src]

        while stack:
            v = stack.pop()
            if not visited[v]:
                print(v, end=' -> ')
                visited[v] = True
                # Only iterate actual neighbors (no wasted 0-checks)
                for neighbor in self.adj[v]:
                    if not visited[neighbor]:
                        stack.append(neighbor)

# Same graph, adjacency list version:
g = GraphList(6)
g.add_edge(0,1); g.add_edge(0,2)
g.add_edge(2,3); g.add_edge(2,4)
g.add_edge(3,5); g.add_edge(4,5)
g.dfs(0)
```

2.5 DFS — Recursive with Adjacency Matrix (Python)

Recursion is cleaner. Each function call IS the stack frame — Python's call stack does the backtracking for you automatically.

```
class GraphRecursive:
    def __init__(self, vertex):
        self.mat = [[0]*vertex for _ in range(vertex)]
        self.size = vertex

    def add_edge(self, src, dest):
        self.mat[src][dest] = 1
        self.mat[dest][src] = 1

    def dfs(self, src):
        # Public method: creates visited array, then calls helper
        visited = [False] * self.size
        self._dfs_helper(src, visited)

    def _dfs_helper(self, v, visited):
        # Base action: print and mark current node
        visited[v] = True
        print(v, end=' -> ')

        # Recurse into every unvisited neighbor
        for i in range(self.size):
            if self.mat[v][i] == 1 and not visited[i]:
                self._dfs_helper(i, visited)  # ← goes DEEP here
                # After this returns, we automatically try next i
                # That is backtracking — handled by Python itself!
```

2.6 DFS — Recursive with Adjacency LIST (Python)

```
class GraphRecursiveList:
    def __init__(self, vertex):
        self.size = vertex
        self.adj = [[] for _ in range(vertex)]

    def add_edge(self, src, dest):
        self.adj[src].append(dest)
        self.adj[dest].append(src)

    def dfs(self, src):
        visited = [False] * self.size
        self._dfs_helper(src, visited)

    def _dfs_helper(self, v, visited):
        visited[v] = True
        print(v, end=' -> ')
        for neighbor in self.adj[v]:        # only real neighbors
            if not visited[neighbor]:
                self._dfs_helper(neighbor, visited)
```

2.7 DFS in C — All Variants

C — Matrix + Stack (Iterative)

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100

int mat[MAX][MAX];   // adjacency matrix
int visited[MAX];    // 0=not visited, 1=visited
int stack[MAX];
int top = -1;
int size;

void push(int v) { stack[++top] = v; }
int pop()        { return stack[top--]; }
int isEmpty()    { return top == -1; }

void add_edge(int src, int dest) {
    mat[src][dest] = 1;
    mat[dest][src] = 1;
}

void dfs_iterative(int src) {
    push(src);
    while (!isEmpty()) {
        int v = pop();
        if (!visited[v]) {
            printf("%d -> ", v);
            visited[v] = 1;
            for (int i = 0; i < size; i++) {
                if (mat[v][i] == 1 && !visited[i])
                    push(i);
            }
        }
    }
```

```
}
```

C — Matrix + Recursion

```
void dfs_recursive(int v) {
    visited[v] = 1;
    printf("%d -> ", v);
    for (int i = 0; i < size; i++) {
        if (mat[v][i] == 1 && !visited[i])
            dfs_recursive(i);   // call stack IS the DFS stack
    }
}

// Call it:
// memset(visited, 0, sizeof(visited));
// dfs_recursive(0);
```

C — Adjacency List + Recursion

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int dest;
    struct Node* next;
} Node;

Node* adj[MAX];   // adj[i] = linked list of neighbors of i
int visited[MAX];

void add_edge(int src, int dest) {
    // Add dest to src's list
    Node* n1 = (Node*)malloc(sizeof(Node));
    n1->dest = dest; n1->next = adj[src]; adj[src] = n1;
    // Add src to dest's list (undirected)
    Node* n2 = (Node*)malloc(sizeof(Node));
    n2->dest = src; n2->next = adj[dest]; adj[dest] = n2;
}

void dfs(int v) {
    visited[v] = 1;
    printf("%d -> ", v);
    Node* curr = adj[v];
    while (curr != NULL) {
        if (!visited[curr->dest])
            dfs(curr->dest);
        curr = curr->next;
    }
}
```

2.8 Full DFS Dry Run — Step by Step

Graph: 0 connected to 1,2 | 2 connected to 3,4 | 3 connected to 5 | 4 connected to 5

```
Graph visual:            Adjacency Matrix (6x6):
  0                        0  1  2  3  4  5
 / \                   0 [0][1][1][0][0][0]
```

```
1   2                   1 [1][0][0][0][0][0]
   / \                  2 [1][0][0][1][1][0]
  3   4                 3 [0][0][1][0][0][1]
   \ /                  4 [0][0][1][0][0][1]
    5                   5 [0][0][0][1][1][0]
```

Step	Action	Stack State (top→)	Visited
0	Push src=0	[0]	[F,F,F,F,F,F]
1	Pop 0, print 0, mark visited, push neighbors 1,2	[1, 2]	[T,F,F,F,F,F]
2	Pop 2, print 2, push neighbors 3,4	[1, 3, 4]	[T,F,T,F,F,F]
3	Pop 4, print 4, push neighbor 5	[1, 3, 5]	[T,F,T,F,T,F]
4	Pop 5, print 5, push neighbor 3 (already check)	[1, 3, 3]	[T,F,T,F,T,T]
5	Pop 3, print 3, no unvisited neighbors	[1, 3]	[T,F,T,T,T,T]
6	Pop 3 again → already visited, skip	[1]	[T,F,T,T,T,T]
7	Pop 1, print 1, no unvisited neighbors	[]	[T,T,T,T,T,T]
8	Stack empty → DONE	—	All True

DFS Output: 0 → 2 → 4 → 5 → 3 → 1

Why does DFS print 2 before 1?
Because 1 and 2 are both pushed onto the stack when we process node 0.
Stack is LIFO (Last In First Out). We push 1 first, then 2.
So 2 is on TOP and gets popped first → DFS visits 2 before 1.
This is why DFS order can feel 'backwards' — it follows LIFO, not insertion order.

2.9 DFS Complexity & When to Use

Metric	Matrix Version	List Version
Time Complexity	$O(V^2)$ — checks every cell	O(V + E) — only real neighbors
Space Complexity	$O(V^2)$ for matrix + O(V) stack	O(V + E) for list + O(V) stack
Best For	Dense graphs (many edges)	Sparse graphs (few edges)

When to Use DFS

Finding if a path EXISTS between two nodes
Detecting cycles in a graph
Topological sorting (ordering tasks with dependencies)
Solving mazes / puzzles
Finding connected components
NOT ideal when you need the SHORTEST path — use BFS for that

3. Breadth-First Search (BFS) — Complete Guide

BFS in One Line
Visit ALL neighbors of the current node BEFORE going deeper.
Think of it as dropping a stone in water — ripples spread outward level by level.

3.1 BFS vs DFS — The Fundamental Difference

```
Same graph:                DFS (goes DEEP first):     BFS (goes WIDE first):
  0                        0 → 1 → 3 → 2 → 4          0 → 1 → 2 → 3 → 4
 /|\                       (dives down one path)       (all neighbors first)
1 2 3
|   |
4   5   (not shown)
```

Property	DFS	BFS
Data Structure	Stack (LIFO)	Queue (FIFO)
Traversal Style	Deep first (one path at a time)	Wide first (all neighbors at a time)
Shortest Path?	NO — just finds A path	YES — finds SHORTEST path (unweighted)
Memory Use	Low (follows one path)	Higher (stores whole frontier)
Mark Visited	When popping	When PUSHING (enqueueing)

Critical BFS Rule: Mark Visited When ADDING to Queue
In DFS, we mark visited when we POP from stack.
In BFS, we must mark visited when we ADD to queue (not when we process).

WHY? Because in BFS multiple nodes at the SAME level can all see the same neighbor. If we don't mark it early, node X can be added to the queue 3 times by 3 different neighbors before it's ever processed. This causes duplicate visits and wrong output.

Rule: BFS = mark when enqueue, DFS = mark when dequeue.

3.2 BFS — Queue Basics (FIFO)

BFS uses a Queue: First In, First Out. We add to the BACK and remove from the FRONT. Python's collections.deque is perfect for this.

```
from collections import deque

queue = deque()
queue.append(1)    # Enqueue — add to BACK:   [1]
queue.append(2)    # Enqueue:                 [1, 2]
```

```
queue.append(3)    # Enqueue:                [1, 2, 3]

queue.popleft()    # Dequeue — remove from FRONT:  returns 1, queue=[2,3]
queue.popleft()    # returns 2, queue=[3]

# Why not a regular list?
# list.pop(0) is O(n) — it shifts every element
# deque.popleft() is O(1) — optimized for front removal
```

3.3 BFS — Iterative with Adjacency Matrix (Python)

```
from collections import deque

class Graph:
    def __init__(self, vertex):
        self.mat = [[0]*vertex for _ in range(vertex)]
        self.size = vertex

    def add_edge(self, src, dest):
        if (0 <= src < self.size) and (0 <= dest < self.size):
            self.mat[src][dest] = 1
            self.mat[dest][src] = 1
        else:
            print('Invalid Edge')

    def bfs(self, src):
        visited = [False] * self.size
        queue = deque([src])    # start: enqueue source
        visited[src] = True     # mark source IMMEDIATELY when enqueuing

        while queue:
            v = queue.popleft()       # dequeue from FRONT (FIFO)
            print(v, end=' -> ')      # process (print) the node

            # Check all possible neighbors
            for i in range(self.size):
                # If edge exists and neighbor not visited
                if self.mat[v][i] == 1 and not visited[i]:
                    visited[i] = True      # mark when ADDING to queue
                    queue.append(i)        # enqueue to BACK

# Usage:
g = Graph(7)
g.add_edge(0,1); g.add_edge(0,2)
g.add_edge(1,2); g.add_edge(1,3)
g.add_edge(2,3); g.add_edge(3,4)
g.add_edge(4,5); g.add_edge(5,6)
g.bfs(0)
# Output: 0 -> 1 -> 2 -> 3 -> 4 -> 5 -> 6
```

3.4 BFS — Iterative with Adjacency LIST (Python)

```
from collections import deque

class GraphList:
```

```
    def __init__(self, vertex):
        self.size = vertex
        self.adj = [[] for _ in range(vertex)]

    def add_edge(self, src, dest):
        self.adj[src].append(dest)
        self.adj[dest].append(src)

    def bfs(self, src):
        visited = [False] * self.size
        queue = deque([src])
        visited[src] = True

        while queue:
            v = queue.popleft()
            print(v, end=' -> ')
            # Only iterate actual neighbors — no wasted zeros
            for neighbor in self.adj[v]:
                if not visited[neighbor]:
                    visited[neighbor] = True
                    queue.append(neighbor)
```

3.5 BFS — Recursive (Python)

BFS is naturally iterative. To make it recursive, we simulate the queue manually and pass it to each recursive call.

```
from collections import deque

class GraphRecursiveBFS:
    def __init__(self, vertex):
        self.mat = [[0]*vertex for _ in range(vertex)]
        self.size = vertex

    def add_edge(self, src, dest):
        self.mat[src][dest] = 1
        self.mat[dest][src] = 1

    def bfs(self, src):
        visited = [False] * self.size
        queue = deque([src])
        visited[src] = True
        self._bfs_helper(queue, visited)

    def _bfs_helper(self, queue, visited):
        if not queue:    # base case: nothing left to process
            return

        v = queue.popleft()       # process front of queue
        print(v, end=' -> ')

        for i in range(self.size):
            if self.mat[v][i] == 1 and not visited[i]:
                visited[i] = True
                queue.append(i)

        self._bfs_helper(queue, visited)  # recurse on remaining queue
```

```
# Note: This is rarely used in practice.
# Iterative BFS is cleaner and avoids recursion limit issues.
# But it proves BFS CAN be written recursively.
```

3.6 BFS in C — All Variants

C — Matrix + Queue (Iterative)

```c
#include <stdio.h>
#include <string.h>
#define MAX 100

int mat[MAX][MAX];
int visited[MAX];
int queue[MAX];
int front = 0, rear = 0;
int size;

void enqueue(int v) { queue[rear++] = v; }
int dequeue()       { return queue[front++]; }
int isEmpty()       { return front == rear; }

void add_edge(int src, int dest) {
    mat[src][dest] = 1;
    mat[dest][src] = 1;
}

void bfs(int src) {
    memset(visited, 0, sizeof(visited));
    enqueue(src);
    visited[src] = 1;   // mark when enqueuing!

    while (!isEmpty()) {
        int v = dequeue();
        printf("%d -> ", v);
        for (int i = 0; i < size; i++) {
            if (mat[v][i] == 1 && !visited[i]) {
                visited[i] = 1;   // mark before enqueuing
                enqueue(i);
            }
        }
    }
}
```

C — Adjacency List + Queue (Iterative)

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX 100

typedef struct Node { int dest; struct Node* next; } Node;

Node* adj[MAX];
int visited[MAX];
```

```
int queue[MAX];
int front = 0, rear = 0;

void enqueue(int v) { queue[rear++] = v; }
int dequeue()       { return queue[front++]; }
int isEmpty()       { return front == rear; }

void add_edge(int src, int dest) {
    Node* n1 = malloc(sizeof(Node));
    n1->dest = dest; n1->next = adj[src]; adj[src] = n1;
    Node* n2 = malloc(sizeof(Node));
    n2->dest = src; n2->next = adj[dest]; adj[dest] = n2;
}

void bfs(int src) {
    memset(visited, 0, sizeof(visited));
    enqueue(src);
    visited[src] = 1;
    while (!isEmpty()) {
        int v = dequeue();
        printf('%d -> ', v);
        Node* curr = adj[v];
        while (curr != NULL) {
            if (!visited[curr->dest]) {
                visited[curr->dest] = 1;
                enqueue(curr->dest);
            }
            curr = curr->next;
        }
    }
}
```

3.7 Full BFS Dry Run — Step by Step

Graph (same as DFS dry run): 0-1, 0-2, 2-3, 2-4, 3-5, 4-5. Starting from node 0.

Step	Action	Queue State (front→back)	Visited
0	Enqueue src=0, mark 0 visited	[0]	[T,F,F,F,F,F]
1	Dequeue 0, print 0. Neighbors: 1(unvisited)→enqueue+mark, 2(unvisited)→enqueue+mark	[1, 2]	[T,T,T,F,F,F]
2	Dequeue 1, print 1. Neighbor: 0 already visited → skip	[2]	[T,T,T,F,F,F]
3	Dequeue 2, print 2. Neighbors: 3,4 unvisited → enqueue+mark both	[3, 4]	[T,T,T,T,T,F]
4	Dequeue 3, print 3.	[4, 5]	[T,T,T,T,T,T]

Step	Action	Queue State (front→back)	Visited
	Neighbors: 2(visited), 5(unvisited)→enqueue+mark		
5	Dequeue 4, print 4. Neighbors: 2(visited), 5(already visited) → skip	[5]	[T,T,T,T,T,T]
6	Dequeue 5, print 5. Neighbors: 3,4 both visited → skip	[]	[T,T,T,T,T,T]
7	Queue empty → DONE	—	All True

BFS Output: 0 → 1 → 2 → 3 → 4 → 5

Notice: BFS visits nodes level by level (0 is level 0; 1,2 are level 1; 3,4 are level 2; 5 is level 3)

3.8 BFS for Shortest Path

BFS naturally finds the shortest path in an unweighted graph because it explores level by level. The FIRST time BFS reaches a node, it has taken the minimum number of steps.

```
def bfs_shortest_path(self, src, dest):
    visited = [False] * self.size
    # Store (node, path_so_far) in queue
    queue = deque([(src, [src])])
    visited[src] = True

    while queue:
        v, path = queue.popleft()

        if v == dest:          # reached destination!
            return path        # this IS the shortest path

        for i in range(self.size):
            if self.mat[v][i] == 1 and not visited[i]:
                visited[i] = True
                queue.append((i, path + [i]))   # extend path

    return None   # no path exists

# Example:
# g.bfs_shortest_path(0, 5) might return [0, 2, 3, 5]
# DFS might also find [0, 2, 4, 5] — same length here, but
# for complex graphs BFS GUARANTEES shortest, DFS does NOT.
```

3.9 BFS Complexity & When to Use

Metric	Matrix Version	List Version
Time Complexity	$O(V^2)$	$O(V + E)$

Metric	Matrix Version	List Version
Space Complexity	$O(V^2)$ matrix + O(V) queue	O(V+E) list + O(V) queue
Best For	Dense graphs	Sparse graphs

When to Use BFS

Finding the SHORTEST path in an unweighted graph
Level-order traversal (visit all nodes at depth 1, then depth 2, etc.)
Finding all nodes within a given distance
Social networks: find friends of friends (degrees of separation)
GPS / mapping: shortest route (unweighted)
Web crawling: visit all pages at distance 1 first, then distance 2...

4. DFS vs BFS — Complete Comparison

Property	DFS	BFS
Full Name	Depth-First Search	Breadth-First Search
Core Idea	Go DEEP first, backtrack later	Go WIDE first, level by level
Data Structure	Stack (LIFO)	Queue (FIFO)
Mark Visited	When POPPING (dequeue)	When PUSHING (enqueue)
Shortest Path	No guarantee	YES — guaranteed (unweighted)
Time (Matrix)	$O(V^2)$	$O(V^2)$
Time (List)	O(V + E)	O(V + E)
Space	O(V) stack	O(V) queue (can be larger)
Memory	Uses less (one path at a time)	Uses more (entire frontier)
Implementations	Stack (iterative) or Recursion	Queue (iterative) or Recursive
Storage Types	Matrix or Adjacency List	Matrix or Adjacency List
Use Cases	Cycles, paths, topological sort, maze	Shortest path, level-order, social networks

4.1 All Variants — Quick Reference

Algorithm	Style	Storage	Notes
DFS	Iterative (Stack)	Adjacency Matrix	Standard class version; easy to trace
DFS	Iterative (Stack)	Adjacency List	More efficient for sparse graphs
DFS	Recursive	Adjacency Matrix	Cleanest code; Python call stack = DFS stack
DFS	Recursive	Adjacency List	Most efficient DFS variant overall
BFS	Iterative (Queue)	Adjacency Matrix	Standard BFS; mark visited on enqueue!
BFS	Iterative (Queue)	Adjacency List	More efficient; preferred in interviews
BFS	Recursive	Adjacency Matrix	Rare; pass queue as param; iterative is better
BFS	Shortest Path	Either	Store (node, path) pairs in queue

4.2 Common Mistakes — Interview Trap List

Mistakes to AVOID

1. Using list.pop(0) in Python for BFS — O(n) cost! Use deque.popleft() which is O(1)
2. Marking visited on POP in BFS — causes same node to be enqueued multiple times
3. Forgetting to mark visited at ALL — causes infinite loop on cyclic graphs
4. Using Graph(6) when you have nodes 0-6 — that needs Graph(7)!
5. Assuming DFS gives shortest path — it does NOT; BFS does
6. Using range(self.size-1) instead of range(self.size) — misses the last node
7. In C: forgetting memset(visited, 0,...) — stale data causes wrong results